

АННОТАЦИЯ.....	1
ВВЕДЕНИЕ.....	2
1. ПОСТАНОВКА ЗАДАЧИ И ТРЕБОВАНИЯ.....	3
1.1 Назначение и цель приложения.....	3
1.2 Функциональные требования.....	3
1.3 Выбор технологий разработки.....	4
2. ПРОЕКТИРОВАНИЕ ПРИЛОЖЕНИЯ.....	5
2.1 Архитектура и структура проекта.....	5
2.2 Модель состояния игровой партии.....	5
2.3 Алгоритм проверки победы.....	6
2.4 Проектирование пользовательского интерфейса.....	6
3. РЕАЛИЗАЦИЯ ОСНОВНЫХ КОМПОНЕНТОВ.....	8
3.1 Реализация экранов на Jetpack Compose.....	8
3.2 Отрисовка доски и обработка касаний.....	9
3.3 Реализация компьютерного соперника.....	9
3.4 Подтверждение хода и отмена действий.....	10
3.5 Локализация, звук и вибрация.....	11
4. ТЕСТИРОВАНИЕ И ОЦЕНКА РЕЗУЛЬТАТОВ.....	12
4.1 Стратегия тестирования.....	12
4.2 Автоматизированные тесты.....	12
4.3 Сценарии приемочного тестирования.....	13
4.4 Результаты и ограничения.....	13
ЗАКЛЮЧЕНИЕ.....	14
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	15
ПРИЛОЖЕНИЕ А. ФРАГМЕНТЫ ПРОГРАММНОГО КОДА.....	16

АННОТАЦИЯ

Настоящий отчет посвящен проектированию и разработке нативного Android-приложения «Гомоку» с русскоязычным пользовательским интерфейсом. Приложение реализовано на языке Kotlin с применением Jetpack Compose и Material 3. Игровая доска размером 15×15 формируется средствами Canvas, а правила партии, управление состоянием и компьютерный соперник реализованы отдельными модулями без использования WebView, HTML и JavaScript. Архитектура разделяет модели, правила, игровой процесс, оценку позиции AI и визуальные компоненты, что упрощает сопровождение и тестирование.

В результате разработаны два режима: локальная игра двух пользователей на одном устройстве и партия против компьютера с выбором цвета. Реализованы предварительный ход с повторным подтверждением, отмена последнего хода или полного раунда, определение победы по четырём направлениям, подсветка выигрышной последовательности, обработка ничьей, звук, вибрация и сохранение интерфейса в безопасных зонах экрана. Алгоритм AI сначала проверяет возможность немедленной победы и необходимость блокировки, затем оценивает атакующие и защитные комбинации в окрестности уже поставленных камней. Девять сценариев логического smoke-теста успешно выполнены, а проект содержит 21 метод модульного тестирования.

Ключевые слова: Android; Kotlin; Jetpack Compose; Material 3; Canvas; гомоку; искусственный интеллект; эвристическая оценка; локализация; модульное тестирование.

ВВЕДЕНИЕ

Мобильные приложения являются удобной средой для реализации учебных игровых проектов, поскольку объединяют алгоритмическую задачу, проектирование интерфейса, обработку событий и особенности жизненного цикла платформы. Игра гомоку хорошо подходит для такой работы: правила понятны пользователю, но программная реализация требует точного сопоставления касаний с координатами доски, устойчивого хранения состояния, проверки нескольких направлений победы и выбора разумного хода компьютерным соперником.

Актуальность работы связана с освоением современного стека нативной Android-разработки. Jetpack Compose позволяет описывать интерфейс декларативно, синхронизируя отображение с состоянием приложения. Canvas даёт прямой контроль над геометрией линий, звёздных точек, камней и подсветки. Kotlin обеспечивает типобезопасные модели и удобные неизменяемые структуры данных, а тесты JUnit позволяют проверять игровую логику отдельно от интерфейса и устройства.

Цель работы — разработать завершённое нативное Android-приложение для игры в гомоку с локальным и компьютерным режимами, русским интерфейсом и средствами обратной связи. Объектом исследования является процесс создания интерактивного мобильного приложения. Предмет исследования — методы моделирования состояния настольной игры, построения Compose-интерфейса, обработки касаний Canvas и эвристического выбора хода.

Для достижения цели поставлены следующие задачи: определить функциональные требования; спроектировать модели игрового состояния; реализовать правила установки камней и проверки пяти в ряд; создать адаптивную отрисовку доски; разработать компьютерного соперника; обеспечить подтверждение и отмену ходов; локализовать интерфейс; реализовать звуковую и тактильную обратную связь; подготовить автоматизированные сценарии проверки.

Практическая значимость результата состоит в том, что проект может быть собран в Android Studio и установлен на устройство начиная с Android 7.0. Приложение полностью автономно: AI и правила работают локально, сетевое подключение во время игры не требуется. Исходный код организован по функциональным модулям и может использоваться как основа для последующего расширения уровней сложности, сохранения партий и сетевого режима.

1. ПОСТАНОВКА ЗАДАЧИ И ТРЕБОВАНИЯ

1.1 Назначение и цель приложения

Разрабатываемое приложение предназначено для проведения партии гоמוку на смартфоне. Пользователь должен иметь возможность быстро начать игру, выбрать режим и цвет, получать однозначную информацию о текущем ходе и завершении партии. Интерфейс ориентирован на портретный экран и управление одним касанием. Все основные действия доступны с игрового экрана, а потенциально разрушительные команды сопровождаются диалогом подтверждения.

В локальном режиме два участника по очереди используют одно устройство. В режиме против компьютера пользователь выбирает чёрные или белые камни. Чёрные всегда ходят первыми; если пользователь выбирает белые, компьютер автоматически размещает первый чёрный камень в центре. Такое поведение соответствует привычной стратегии открытия и устраняет неопределённость пустой позиции для AI.

Приложение должно восприниматься как самостоятельный нативный продукт. Поэтому интерфейс строится средствами Compose, игровое поле рисуется Canvas, а текст хранится в Android string resources. Фон, аватары и значок приложения размещаются в ресурсах APK. Это обеспечивает работу без локального веб-сервера и исключает зависимость игрового процесса от браузерного движка.

1.2 Функциональные требования

Основные требования сгруппированы в таблице 1.1. При проектировании учитывались не только правила игры, но и защита пользователя от случайного действия, доступность состояния партии и предсказуемое поведение отмены.

№	Требование	Критерий выполнения
1	Доска 15×15	Камни устанавливаются только на свободные пересечения
2	Два режима игры	Поддерживаются локальная партия и игра против AI
3	Выбор цвета	В режиме AI пользователь выбирает чёрные или белые
4	Проверка результата	Определяются горизонталь, вертикаль и две диагонали
5	Подтверждение хода	Первое касание создаёт preview, повторное фиксирует ход
6	Отмена	В локальном режиме снимается один ход, в AI — полный раунд
7	Обратная связь	Показываются очередь, последний ход, победная линия, звук и вибрация
8	Русская локализация	Все пользовательские сообщения находятся в strings.xml

К нефункциональным требованиям отнесены автономная работа, плавность интерфейса, отсутствие смещения доски при изменении состояния, поддержка системных безопасных зон и возможность тестировать правила без запуска Android UI. Логика не должна зависеть от конкретного размера экрана,

поэтому координаты рассчитываются относительно фактического размера Canvas.

1.3 Выбор технологий разработки

Технологический стек выбран с учётом требований к нативности, современному интерфейсу и разделению логики. Минимальная версия платформы установлена на уровне API 24, целевая и компилируемая версия — Android 36.1. Исходный код использует Java 11 как уровень совместимости байткода.

Компонент	Используемая технология	Назначение
Язык	Kotlin 2.2.10	Модели, правила, AI и UI-логика
Интерфейс	Jetpack Compose	Декларативное построение экранов
Дизайн-система	Material 3	Диалоги, переключатели и базовые компоненты
Графика	Compose Canvas	Сетка, камни, отметки и победная линия
Состояние	rememberSaveable	Сохранение выбранного режима и экрана
Тестирование	JUnit 4	Проверка правил и AI без устройства
Сборка	Gradle Kotlin DSL	Конфигурация зависимостей и APK

Jetpack Compose уменьшает количество связующего кода между представлением и состоянием. При изменении объекта GomokuState необходимые компоненты автоматически перерисовываются. При этом игровой алгоритм остаётся обычным Kotlin-кодом и не содержит зависимостей от Composable-функций. Такое разделение повышает тестируемость и позволяет изменять визуальный слой без переписывания правил.

2. ПРОЕКТИРОВАНИЕ ПРИЛОЖЕНИЯ

2.1 Архитектура и структура проекта

Проект реализован как одно Android-приложение с пакетом `com.example.gomoku`. Архитектура построена по принципу разделения ответственности. Модель состояния хранит только данные партии; функции правил работают с доской; игровой модуль выполняет переходы состояния; AI выбирает координату; Compose-компоненты отображают готовое состояние и передают действия пользователя.

Файл	Ответственность
GameModels.kt	Константы, режимы, координаты, ходы и GomokuState
GameRules.kt	Границы доски, неизменяемое обновление и поиск пяти камней
GomokuGame.kt	Новая партия, preview, установка, отмена и перезапуск
AiEvaluator.kt	Распознавание и численная оценка комбинаций
AiPlayer.kt	Формирование кандидатов и выбор лучшего хода
GomokuBoard.kt	Canvas-отрисовка и преобразование касаний в клетки
GameScreen.kt	Экран партии, задержка AI и системная обратная связь
GameComponents.kt	Карточки игроков, панели, настройки и диалоги
MainActivity.kt	Точка входа, домашний экран и выбор режима

Поток данных является однонаправленным. Пользовательское касание преобразуется в координату `BoardPoint` и передаётся в `GomokuGame.tapCell`. Функция возвращает новый `GomokuState`. Экран получает новое состояние и перерисовывает только зависимые элементы. После подтверждённого человеческого хода в режиме AI экран временно блокирует доску, ждёт около 500 мс и запрашивает ход `AiPlayer`.

Неизменяемые списки используются осознанно. Обновление клетки создаёт новую строку и новый внешний список только для изменяемой позиции, не меняя предыдущий объект. Это соответствует модели Compose и упрощает сравнение состояний в тестах. История `moves` служит единственным источником для восстановления доски после отмены полного раунда.

2.2 Модель состояния игровой партии

Центральная структура `GomokuState` содержит доску, текущего игрока, историю, предварительный ход, победителя, выигрышные клетки, признак ничьей и настройки режима. Вычисляемые свойства `gameOver`, `lastMove`, `aiPlayer` и `isHumanTurn` не дублируют данные, а выводятся из базовых полей. Благодаря этому исключаются противоречивые комбинации, например наличие победителя при продолжающейся партии.

Поле	Тип	Смысл
board	List<List<Int>>	Состояние 225 пересечений
currentPlayer	Int	BLACK или WHITE
moves	List<Move>	Хронология подтверждённых ходов
pendingMove	BoardPoint?	Полупрозрачный предварительный камень
winner	Int?	Победитель или null
winningCells	List<BoardPoint>	Пять клеток для визуальной подсветки
isDraw	Boolean	Признак заполненной доски без победы

Камни кодируются целыми константами EMPTY, BLACK и WHITE. Размер поля задаётся единственной константой BOARD_SIZE = 15, что предотвращает расхождение между правилами, AI и Canvas. Ход хранит строку, столбец и игрока. Координаты предварительного действия представлены отдельным типом BoardPoint, поскольку preview ещё не является подтверждённым Move.

Начальное состояние зависит от режима и цвета. В локальной партии доска пуста, текущий игрок — чёрный. В партии против компьютера при выборе белых вызывается placeStone для центральной точки (7, 7); затем очередь автоматически переходит пользователю. Центральный дебют защищён от отмены, чтобы после undo не возникло пустой позиции, в которой первым должен снова ходить AI.

2.3 Алгоритм проверки победы

После каждого подтверждённого хода проверяются четыре независимых направления: горизонтальное (0, 1), вертикальное (1, 0), нисходящая диагональ (1, 1) и восходящая диагональ (1, -1). Для каждого направления алгоритм собирает непрерывную последовательность одинаковых камней сначала в отрицательную, затем в положительную сторону от последнего хода.

Проверка начинается только для корректной координаты и непустой клетки. Функция collectLine останавливается на границе, пустой клетке или камне другого цвета. Если длина линии не меньше пяти, выбирается последовательность из пяти точек, обязательно содержащая последний ход. Для линии длиной более пяти стартовое смещение выбирается около последнего хода, поэтому визуальная подсветка остаётся стабильной и понятной.

$$W = \{p_0, p_1, p_2, p_3, p_4\}, \quad p_i = p_0 + i \cdot (\Delta r, \Delta c)$$

Временная сложность проверки одного хода ограничена длиной четырёх линий на доске 15×15 и практически постоянна. Ничья устанавливается, если победитель не найден, а количество подтверждённых ходов равно 225. После победы или ничьей поле gameOver блокирует дальнейшие изменения через tapCell и placeStone.

2.4 Проектирование пользовательского интерфейса

Интерфейс состоит из домашнего и игрового экранов. На домашнем экране размещаются название, подзаголовок, кнопки локальной игры, игры с компьютером и правил, а также управление звуком. При выборе AI открывается диалог выбора цвета. После выбора создаётся соответствующее состояние, и корневая функция переключает содержимое на GameScreen.

Игровой экран разделён на верхнюю область игроков, центральную доску, текст текущего состояния и нижнюю панель действий. Карточки чёрных и белых показывают имя, аватар, образец камня и метку AI. Активная карточка

подсвечивается. Во время расчёта компьютерного хода выводится сообщение «Компьютер думает...», а обработка касаний временно отключается.

Для защиты от ошибочного действия выход и перезапуск требуют подтверждения. Результат партии выводится в отдельном диалоге с возможностью сыграть ещё раз, посмотреть доску или перейти на главную. Настройки подтверждения хода, звука и вибрации доступны без выхода из текущей партии. Safe area учитывается через `statusBarsPadding` и `navigationBarsPadding`.

Визуальная концепция использует светлый пейзажный фон, тёмно-зелёную доску, золотистые линии и объёмные чёрные и белые камни. Последний ход отмечается красной точкой, а выигрышные камни и соединяющая линия получают дополнительную подсветку. Размеры игрового поля заданы устойчиво, поэтому изменение текста, анимации или хода не сдвигает сетку.

3. РЕАЛИЗАЦИЯ ОСНОВНЫХ КОМПОНЕНТОВ

3.1 Реализация экранов на Jetpack Compose

MainActivity наследуется от ComponentActivity и вызывает setContent. Тема GomokuTheme устанавливает палитру Material 3, после чего функция GomokuRoot управляет экраном приложения. Значения активного режима, выбранного цвета и видимости диалогов хранятся через rememberSaveable, поэтому допускают восстановление при пересоздании Activity.

Compose-функции разделены на небольшие компоненты: HomeScreen, PlayerHeader, PlayerCard, BottomActions, SettingsDialog, ResultDialog и другие. Параметры передаются явно, а события представлены лямбда-функциями. Такая композиция позволяет переиспользовать одинаковые элементы в локальном и AI-режимах без копирования разметки.

Тексты не записаны непосредственно в интерфейсных функциях. Они извлекаются через stringResource из strings.xml. Это решение облегчило перевод на русский язык и позволило подобрать размеры диалогов под длинные подписи. В production-коде сохранена только короткая техническая метка AI, одинаково понятная для выбранной локали.

Ресурс game_scene.jpg используется как фоновое оформление. Он не содержит игровых правил и может быть заменён без изменений Kotlin-кода. Аватары игроков также вынесены в drawable-nodpi, что исключает нежелательное масштабирование плотностью экрана и позволяет самостоятельно контролировать размер в Compose.



Рисунок 3.1 — Фоновый ресурс игрового интерфейса

3.2 Отрисовка доски и обработка касаний

GomokuBoard строится на Compose Canvas. Фактический размер квадратной области определяется во время отрисовки. Внутренний отступ отделяет крайние пересечения от декоративной рамки, а шаг сетки вычисляется как отношение доступной стороны к `BOARD_SIZE - 1`. Те же параметры используются для линий, звёздных точек, камней и преобразования касаний.

$$step = (side - 2 \cdot padding) / (BOARD_SIZE - 1)$$

Координата касания переводится в ближайшие целые `row` и `col` округлением относительно начала сетки. Затем `isInsideBoard` проверяет диапазон, а игровой модуль проверяет занятость клетки. Камень рисуется точно в точке `padding + index \cdot step`, поэтому визуальное положение и логическая координата используют одну систему и не расходятся при масштабировании.

Чёрные и белые камни формируются несколькими кругами с градиентами, бликами и тенями. Для предварительного хода применяется пониженная прозрачность. Появление подтверждённого камня анимируется масштабированием, но геометрический центр остаётся неизменным. Отметка последнего хода рисуется поверх камня и не участвует в обработке события.

После победы Canvas получает `winningCells`. Центры первой и последней клетки соединяются линией, а все пять камней дополнительно выделяются. Такой подход не требует вычислять угол отдельно: направление уже представлено координатами победной последовательности.

3.3 Реализация компьютерного соперника

Компьютерный соперник реализован как локальный детерминированный по оценке алгоритм с небольшим случайным выбором среди равных кандидатов. Он не использует сеть и не выполняет полный перебор дерева игры. Это обеспечивает быстрое решение на мобильном устройстве и предсказуемую задержку интерфейса.

Сначала формируется множество свободных клеток в радиусе двух позиций от каждого установленного камня. На пустой доске единственным кандидатом является центр. Ограничение области поиска существенно уменьшает число оцениваемых ходов: вместо всех 225 клеток рассматриваются только локально значимые точки рядом с текущими комбинациями.

Приоритеты выбора заданы явно. На первом шаге AI ищет собственный немедленный выигрыш. На втором — обязательный блок победного хода соперника. Только при отсутствии таких вариантов вычисляется суммарная эвристическая оценка атаки, защиты, близости к камням и центра доски.

$$Score(p) = 100 \cdot Attack(p) + 112 \cdot Defense(p) + Locality(p) + Center(p)$$

Защитный коэффициент 112 немного выше атакующего 100, поэтому соперник не игнорирует опасные комбинации пользователя. При этом проверка немедленной победы выполняется раньше блокировки, что сохраняет правильный игровой приоритет: компьютер завершает собственную пятёрку, если она доступна.

Комбинация	Количество открытых концов	Оценка
Пять и более	любое	1 000 000
Открытая четвёрка	2	120 000
Закрытая четвёрка	1	25 000
Открытая тройка	2	8 000
Закрытая тройка	1	1 200
Открытая двойка	2	450
Закрытая двойка	1	100
Одиночный камень	2	20

AiEvaluator анализирует четыре направления так же, как модуль победы. Для каждой стороны от кандидата подсчитываются непрерывные камни и проверяется свободный конец. Если как минимум два направления имеют уровень открытой тройки или выше, добавляется бонус двойной угрозы 18 000. Среди ходов с одинаковой максимальной оценкой выбирается случайный, поэтому партии не повторяются полностью.

Расчёт AI запускается после хода человека в coroutine-эффекте экрана. Задержка около 500 мс делает действие визуально понятным. Флаг `isAiThinking` блокирует доску и команду отмены на время вычисления, предотвращая конкурентное изменение состояния.

3.4 Подтверждение хода и отмена действий

Подтверждение хода реализовано внутри `GomokuGame.tapCell`. Если настройка выключена, касание сразу вызывает `placeStone`. Если подтверждение включено, первое касание свободной клетки записывает `BoardPoint` в `pendingMove`, не изменяя `board` и `moves`. Повторное касание той же клетки фиксирует камень. Касание другой свободной точки переносит `preview`.

Такой механизм не требует отдельных кнопок «Подтвердить» и «Отмена». Пользователь взаимодействует непосредственно с доской, а полупрозрачный камень ясно показывает ожидаемый результат. Касание занятой клетки не удаляет текущий `preview`, что предотвращает случайную потерю выбранной позиции.

В локальном режиме `undo` снимает один последний камень и возвращает очередь игроку, который его поставил. Одновременно очищаются `winner`, `winningCells` и `isDraw`, поэтому можно продолжить партию даже после показа результата. Предварительный, но не подтверждённый ход просто очищается и не затрагивает историю.

В режиме против AI отмена учитывает понятие полного раунда. Если компьютер уже ответил, удаляются два последних хода; если ответ ещё не выполнен, удаляется только ход пользователя. Для игры белыми автоматический центральный ход AI защищён: его нельзя отменить отдельно. Доска восстанавливается свёрткой оставшейся истории moves, что проще и надёжнее последовательного изменения нескольких клеток.

3.5 Локализация, звук и вибрация

Русская локализация охватывает домашний экран, карточки игроков, действия, настройки, правила, сообщения результата и подсказки текущего состояния. Строки вынесены в ресурсный файл strings.xml, поэтому Compose-компоненты получают текст через идентификаторы. Это также предотвращает смешение языков при дальнейшем сопровождении.

Звуковая обратная связь формируется средствами платформы после подтверждённого хода. Настройка звука независима от вибрации. Тактильный отклик вызывается только для окончательной установки камня, а не для предварительного preview. Такое разделение снижает количество лишних сигналов и делает двухэтапное подтверждение различимым.

Интерфейс учитывает длину русских фраз: кнопки имеют ограничения ширины, диалоги допускают перенос строк, а компактные подписи нижней панели масштабируются в пределах стабильных контейнеров. Системные отступы предотвращают перекрытие содержимого строкой состояния, камерой или навигационной панелью.

4. ТЕСТИРОВАНИЕ И ОЦЕНКА РЕЗУЛЬТАТОВ

4.1 Стратегия тестирования

Тестирование разделено на три уровня. На первом уровне проверяются чистые Kotlin-функции: координаты, победа, переходы состояния и AI. На втором уровне анализируются сценарии режима человек–компьютер, где важны автоматический первый ход и особая отмена. На третьем уровне выполняется ручная проверка интерфейса на устройстве или эмуляторе: касания, диалоги, безопасные зоны, звук и вибрация.

Чистая логика не зависит от Android Context и может компилироваться отдельно. Для текущего отчёта GameModels.kt, GameRules.kt, GomokuGame.kt, AiEvaluator.kt, AiPlayer.kt и LogicSmokeTest.kt были собраны Android Studio Kotlin compiler. Выполнены девять smoke-сценариев; результат — «Logic smoke tests passed: 9 scenarios».

Проект дополнительно содержит 21 метод с аннотацией @Test в трёх JUnit-классах. Они документируют ожидаемое поведение preview, прямого хода, отмены, перезапуска, четырёх направлений победы, открытия AI, принудительного блока и развития открытой тройки.

4.2 Автоматизированные тесты

Группа	Количество	Проверяемые свойства
GomokuGameTest	12	Preview, подтверждение, undo, restart, 4 направления победы
AiPlayerTest	4	Центр, немедленная победа, блок, развитие открытой тройки
HumanVsAiGameTest	5	Выбор цвета, первый ход, отмена раунда, restart
LogicSmokeTest	9 сценариев	Сквозная проверка чистой логики; выполнено успешно

Тест firstTapCreatesPreviewWithoutPlacingStone подтверждает, что первое касание не изменяет board и moves. tappingSamePreviewCellCommitsMoveAndChangesPlayer проверяет фиксацию и смену очереди. Четыре теста победы используют горизонтальные, вертикальные и обе диагональные координаты, после чего ожидают winner = BLACK и ровно пять winningCells.

AI-тесты используют фиксированный Random(0), поэтому выбор среди равных оценок воспроизводим. Для немедленной победы и блока допускаются обе свободные стороны открытой четвёрки. Сценарий открытой тройки требует, чтобы AI продолжил собственную линию, а не выбрал удалённую позицию.

Регрессионные тесты человек–AI особенно важны для истории. Они подтверждают, что выбор белых создаёт чёрный камень в центре и передаёт ход человеку; отмена до ответа AI удаляет один ход; отмена после ответа удаляет пару; центральное открытие не снимается; restart сохраняет режим, цвет и настройку подтверждения.

4.3 Сценарии приемочного тестирования

Для проверки пользовательского поведения подготовлен перечень сценариев, приведённый в таблице 4.2. Они выполняются на портретном эмуляторе и физическом устройстве с включёнными системными панелями.

№	Действие	Ожидаемый результат
1	Запуск приложения	Открывается русскоязычный домашний экран
2	Начать локальную игру	Чёрные активны, доска пуста
3	Коснуться свободной точки дважды	Preview превращается в камень в центре пересечения
4	Собрать пять камней	Подсветка линии и диалог результата
5	Выбрать AI и белые	Компьютер начинает ходом (7, 7)
6	Нажать undo после ответа AI	Удаляется полный раунд и возвращается ход пользователя
7	Отключить подтверждение	Следующий камень ставится одним касанием
8	Отключить звук или вибрацию	Каждый канал обратной связи меняется независимо
9	Перезапустить или выйти	Показывается подтверждение, состояние очищается после согласия
10	Проверить разные высоты экрана	Доска и кнопки не перекрывают системные панели

Особое внимание уделяется точности касания: визуальный камень должен иметь тот же центр, что и пересечение, независимо от масштаба Canvas. Для граничных строк и столбцов проверяется невозможность выхода координаты за диапазон 0...14. Быстрые повторные касания во время хода AI не должны изменять состояние.

4.4 Результаты и ограничения

Реализованный проект удовлетворяет сформулированным требованиям. Чистая логика успешно прошла девять выполняемых smoke-сценариев. Наличие 21 модульного теста обеспечивает покрытие критических переходов состояния и решений AI. Русский текст централизован в ресурсах, а Canvas использует единую геометрию для отрисовки и обработки касаний.

Эвристический AI не является полным алгоритмом minimax и не гарантирует оптимальную игру во всех позициях. Его преимущество — низкая вычислительная стоимость и понятные приоритеты. Радиус кандидатов два достаточен для текущего уровня сложности, однако в редких стратегических позициях компьютер может не учитывать далёкий подготовительный ход.

Другим ограничением является отсутствие постоянного сохранения партии после полного завершения процесса. rememberSaveable помогает при пересоздании интерфейса, но история не записывается в базу данных. Также отсутствуют сетевой режим, выбор размера доски, уровни сложности и статистика побед.

Дальнейшее развитие может включать ViewModel, хранилище Room, несколько профилей AI, журнал ходов и сетевую синхронизацию. Модульная архитектура позволяет добавлять эти функции без изменения базовых правил.

ЗАКЛЮЧЕНИЕ

В ходе практики разработано нативное Android-приложение «Гомоку» на Kotlin и Jetpack Compose. Реализована доска 15×15, два режима игры, выбор цвета, двухэтапное подтверждение хода, отмена с учётом режима, перезапуск, выход, правила, настройки и диалог результата. Победа определяется по горизонтали, вертикали и двум диагоналям; выигрышная последовательность сохраняется для визуальной подсветки.

Пользовательский интерфейс построен декларативно и полностью локализован на русский язык. Canvas обеспечивает масштабируемую отрисовку сетки и камней с единым расчётом координат. Системные безопасные зоны, устойчивые размеры компонентов, звук и вибрация улучшают качество мобильного взаимодействия.

Компьютерный соперник реализован локально. Алгоритм формирует кандидатов рядом с существующими камнями, сначала выбирает победу или обязательный блок, затем сравнивает атакующие и защитные шаблоны. Такой подход обеспечивает быстрый ответ без сетевых сервисов и подходит для учебного мобильного приложения.

Практическая проверка чистой логики завершилась успешным прохождением девяти smoke-сценариев; проект содержит 21 модульный тест. Поставленная цель достигнута. Полученный результат демонстрирует применение Kotlin, Compose, Canvas, управления состоянием, локализации и тестирования в одном завершённом проекте и готов к сборке APK в Android Studio.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Android Developers. Jetpack Compose overview. URL: <https://developer.android.com/compose> (дата обращения: 13.07.2026).
2. Android Developers. Graphics in Compose. URL: <https://developer.android.com/develop/ui/compose/graphics/draw/overview> (дата обращения: 13.07.2026).
3. Android Developers. State and Jetpack Compose. URL: <https://developer.android.com/develop/ui/compose/state> (дата обращения: 13.07.2026).
4. Android Developers. App resources: localization. URL: <https://developer.android.com/guide/topics/resources/localization> (дата обращения: 13.07.2026).
5. Android Developers. Haptics design principles. URL: <https://developer.android.com/develop/ui/views/haptics/haptics-principles> (дата обращения: 13.07.2026).
6. Kotlin Documentation. Data classes. URL: <https://kotlinlang.org/docs/data-classes.html> (дата обращения: 13.07.2026).
7. Kotlin Documentation. Collections overview. URL: <https://kotlinlang.org/docs/collections-overview.html> (дата обращения: 13.07.2026).
8. JUnit 4. A programmer-oriented testing framework for Java. URL: <https://junit.org/junit4/> (дата обращения: 13.07.2026).
9. Material Design 3. Components. URL: <https://m3.material.io/components> (дата обращения: 13.07.2026).
10. Исходный код проекта «Гомоку», версия 1.1: Kotlin-модули GameModels, GameRules, GomokuGame, AiEvaluator, AiPlayer и Compose-компоненты, 2026.

ПРИЛОЖЕНИЕ А. ФРАГМЕНТЫ ПРОГРАММНОГО КОДА

А.1 Проверка победной последовательности

```
fun findWinningCells(board: List<List<Int>>, row: Int, col: Int): List<BoardPoint> {
    if (!isInsideBoard(row, col)) return emptyList()
    val player = board[row][col]
    if (player == EMPTY) return emptyList()

    WIN_DIRECTIONS.forEach { (deltaRow, deltaCol) ->
        val line = collectLine(board, row, col, deltaRow, deltaCol, player)
        if (line.size >= 5) {
            return selectFiveContainingMove(line, BoardPoint(row, col))
        }
    }
    return emptyList()
}
```

А.2 Подтверждение хода

```
fun tapCell(state: GomokuState, row: Int, col: Int): GomokuState {
    if (state.gameOver || !isInsideBoard(row, col) || state.board[row][col] != EMPTY) {
        return state
    }
    if (!state.confirmBeforeMove) return placeStone(state, row, col)

    val nextPoint = BoardPoint(row, col)
    return if (state.pendingMove == nextPoint) {
        placeStone(state, row, col)
    } else {
        state.copy(pendingMove = nextPoint)
    }
}
```

А.3 Выбор хода компьютерного соперника

```
val winningMoves = candidates.filter { point ->
    AiEvaluator.isWinningMove(state.board, point, aiPlayer)
}
if (winningMoves.isNotEmpty()) return winningMoves.random(random)

val forcedBlocks = candidates.filter { point ->
    AiEvaluator.isWinningMove(state.board, point, opponent)
}
if (forcedBlocks.isNotEmpty()) return forcedBlocks.random(random)

val score = attack.toLong() * ATTACK_WEIGHT +
    defense.toLong() * DEFENSE_WEIGHT + locality + center
```